
Mini Project 2 – Simple MIPS CPU

Mini Project 2 – COMPENG 4DV4

Chris Hoftyzer 400410904

The design for the MIPS CPU has the following high level state machine structure:

- **S_IDLE:** State will move to Idle in case of a reset, and remain until it is cleared
- **S_FETCH:** Fetch current instruction from instruction memory
- **S_DECODE:** Extract s1, s2, s3, imm, opcode fields from instruction, set control flags, fetch operands from register file
- **S_EXU:** perform arithmetic operations. In the case of lw/sw instructions, the ALU generates addresses, and loads/stores occur in the Execute stage.
- **S_WB:** Write results to register file if necessary
- **S_TERMINATE:** eof instructions and overflows are detected in Decode and Execute respectively, when these are detected, the CPU moves to Terminate state and remains in this state until reset

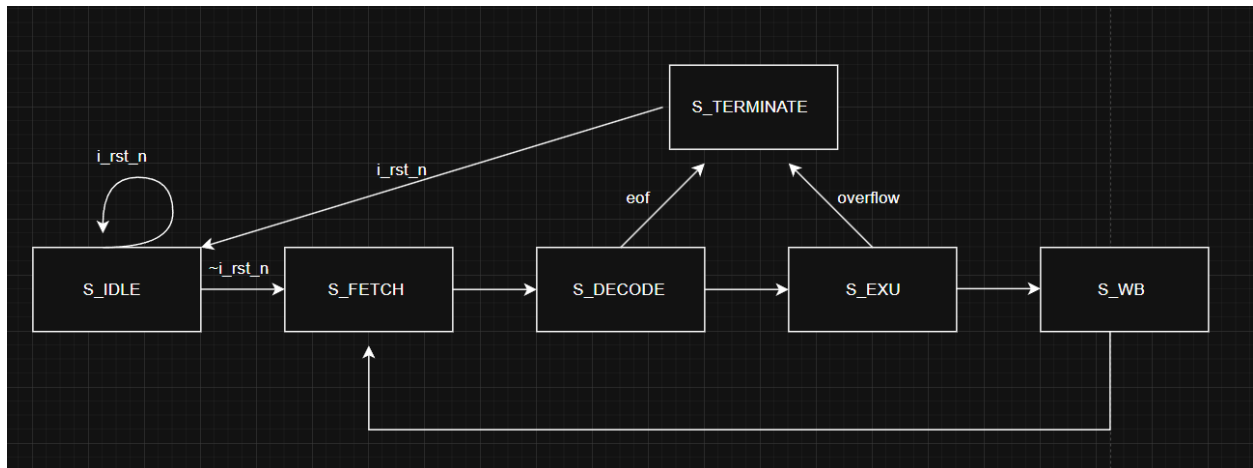


Figure 1: CPU state machine structure

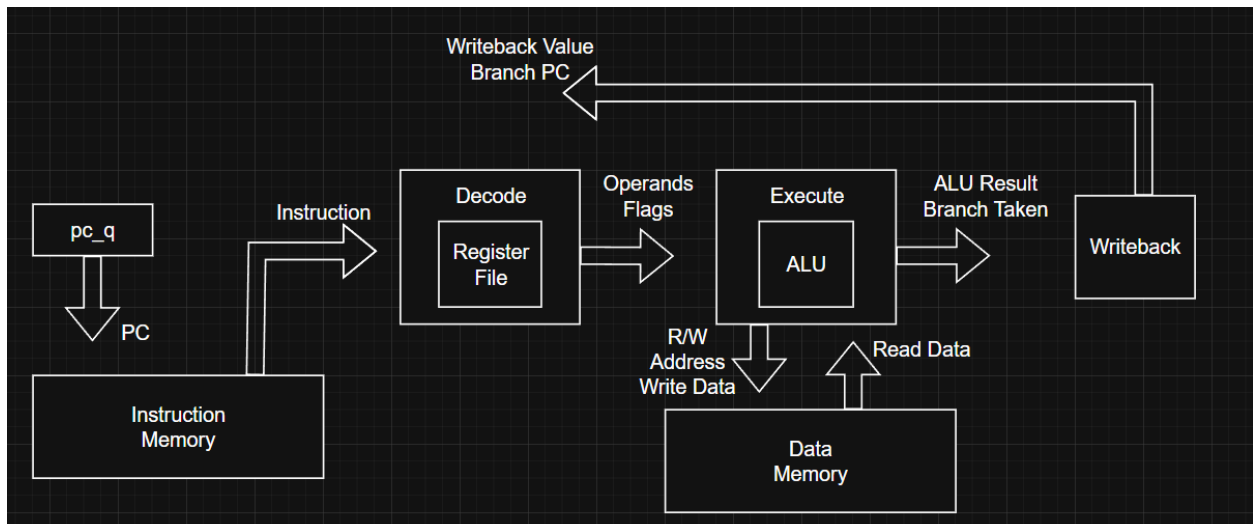


Figure 2: CPU block diagram

Two submodules were instantiated in the design, alu.v and register_file.v, with the following interfaces:

alu.v Interface:

- **i_opr_a[31:0]:** operand a
- **i_opr_b[31:0]:** operand b
- **i_op[5:0]:** opcode
- **o_res[31:0]:** alu result
- **o_overflow:** overflow during alu execution

register_file.v interface:

- **i_clk:** clock
- **i_rst_n:** active low reset – clears register file
- **i_a_rd_addr[4:0]:** read address a
- **i_b_rd_addr[4:0]:** read address b
- **o_a_rd_data[31:0]:** read data a
- **o_b_rd_data[31:0]:** read data b
- **i_wr_en:** write enable
- **i_wr_addr[4:0]:** write address
- **i_wr_data[31:0]:** write data

S_IDLE:

```
end else begin
  case (state)
    S_IDLE: begin
      state <= S_FETCH;
    end
  end case
end
```

Figure 3: S_IDLE sequential logic

The code for the S_IDLE state is very simple, just a few lines of sequential logic. This code is placed in the non reset case of the always_ff block, and advances the state to S_FETCH when reset is high (off).

S_FETCH:

```
end
S_FETCH: begin
    o_status      <= 2'd0;
    o_status_valid <= 1'b0;

    state         <= S_DECODE;
end
S_DECODE: begin
```

Figure 4: S_FETCH sequential logic

```
case (state)
    S_FETCH: begin
        o_i_addr = pc_q;
    end
    S_DECODE: begin
```

Figure 5: S_FETCH combinational logic

The code for S_FETCH is also quite simple. Any status output registers set during previous instructions are cleared, and the program counter is given as the instruction memory read address. The state is always advanced to S_DECODE on the following clock cycle.

S_DECODE:

```

S_DECODE: begin
    if (eof) begin
        o_status      <= 2'd3;
        o_status_valid <= 1'b1;

        state         <= S_TERMINATE;
    end else begin
        opcode_q       <= opcode;

        i_inst_q       <= i_inst;
        r_inst_q       <= r_inst;
        imm_q          <= imm;

        branch_inst_q  <= branch_inst;

        s1_q           <= s1;

        rf_a_rd_data_q <= rf_a_rd_data;
        rf_b_rd_data_q <= rf_b_rd_data;

        state          <= S_EXU;
    end
end

```

Figure 6: S_DECODE sequential logic

```

end
S_DECODE: begin
    opcode      = i_i_inst[31:26];

    i_inst      = (opcode == 6'd3) | (opcode == 6'd4) | (opcode == 6'd5) | (opcode == 6'd9) | (opcode == 6'd10);
    r_inst      = (opcode == 6'd1) | (opcode == 6'd2) | (opcode == 6'd6) | (opcode == 6'd7) | (opcode == 6'd8) | (opcode == 6'd11);
    eof         = (opcode == 6'd12);
    branch_inst = (opcode == 6'd9) | (opcode == 6'd10);

    s3          = i_i_inst[20:16];
    s2          = i_i_inst[25:21];
    s1          = i_inst ? i_i_inst[20:16] : i_i_inst[15:11];
    imm         = i_i_inst[15:0];

    sw_inst     = (opcode == 6'd5);

    rf_a_rd_addr = s2;
    rf_b_rd_addr = sw_inst ? s1 : s3;
end

```

Figure 7: S_DECODE combinational logic

In the decode stage, the opcode is first extracted from the instruction. This opcode is used to set several flags: `i_inst`, `r_inst`, `eof`, `branch_inst` and `sw_inst`, which are used to extract `s3`, `s2`, `s1` and `imm` fields from the instruction. These `s1`, `s2`, `s3` and flags are used to fetch operands from the correct register file address.

If the `eof` flag is set, then state is advanced to `S_TERMINATE`, status output is valid and updated to `2'd3`, and all combinational values are discarded. If it is not set, the state is advanced to `S_EXU`, and flags that are needed for further instruction execution are placed in registers.

S_EXU:

```

end
S_EXU: begin
    if (overflow) begin
        o_status      <= 2'd2;
        o_status_valid <= 1'b1;

        state         <= S_TERMINATE;
    end else begin
        branch_taken_q <= branch_taken;
        alu_res_q      <= alu_res;

        state         <= S_WB;
    end
end
end

```

Figure 8: S_EXU Sequential logic

```

S_EXU: begin
    alu_opr_a    = branch_inst_q ? pc_q + 32'd4 : rf_a_rd_data_q;
    alu_opr_b    = r_inst_q ? rf_b_rd_data_q : {{(DATA_W-16){1'b0}}, imm_q};
    alu_op       = opcode_q;

    branch_taken = ((opcode_q == 6'd9) & (rf_a_rd_data_q == rf_b_rd_data_q)) |
                  ((opcode_q == 6'd10) & (rf_a_rd_data_q != rf_b_rd_data_q));

    lw_inst      = (opcode_q == 6'd4);
    sw_inst      = (opcode_q == 6'd5);

    if (lw_inst) begin
        o_d_wen    = 1'b0;
        o_d_addr   = {2'b00, alu_res[DATA_W-1:2]};
    end else if (sw_inst) begin
        o_d_wen    = 1'b1;
        o_d_addr   = {2'b00, alu_res[DATA_W-1:2]};
        o_d_wdata   = rf_b_rd_data_q;
    end
end
end

```

Figure 9: S_EXU Combinational logic

In the execute stage, math results, new pc values, and memory addresses are calculated in the ALU. ALU operand a is either the next sequential program counter or data from read from register file port a. Alu operand b is either data read from register file port b or an immediate value which may be shifted.

For load and store instructions, addresses are computed by the ALU, and load/store data goes to (load)/from (store) the register file. Branches are resolved outside of the ALU, and the new program counter is computed in the ALU.

Overflows are detected by the ALU. In the case of an overflow, the status is valid with a value of 2'd2, and the state is advanced to S_TERMINATE. Otherwise, state is advanced to S_WB and the ALU and branch taken results are stored in registers.

S_WB:

```
end
S_WB: begin
    lw_inst      = (opcode_q == 6'd4);
    sw_inst      = (opcode_q == 6'd5);

    nxt_pc       = branch_taken_q ? alu_res_q : pc_q + 32'd4;

    rf_wr_en     = ~(branch_inst_q | sw_inst);
    rf_wr_addr   = s1_q;
    rf_wr_data   = lw_inst ? i_d_rdata : alu_res_q;
end
```

Figure 10: S_WB sequential logic

```
end
S_WB: begin
    o_status     <= r_inst_q ? 2'd0 : 2'd1;
    o_status_valid <= 1'b1;

    pc_q         <= nxt_pc;

    state        <= S_FETCH;
end
```

Figure 11: S_WB combinational logic

In the writeback stage, data is written to the register file for all instructions apart from branches and stores. The data written back is either memory output for loads, and the ALU result for all other writeback instructions.

The state is always advanced to fetch to begin executing a new instruction. To finish an instruction, status is valid with a value of 2'd0 or 2'd1 for R and I instructions respectively. The program counter is either incremented or updated to a new branch address from the ALU depending on whether a branch is taken.

S_TERMINATE:

```
end
S_TERMINATE: begin
    o_status     <= 2'd0;
    o_status_valid <= 1'b0;
end
```

Figure 12: S_TERMINATE sequential logic

The S_TERMINATE state is very simple. The status value and valid registers are cleared – previously they would show valid eof or overflow values. When in S_TERMINATE, the state is no longer updated, and the CPU must be reset to return to S_IDLE.

Register File Module:

```
module register_file #(
    parameter DATA_W = 32
)(
    input          i_clk,
    input          i_rst_n,

    input [4:0]     i_a_rd_addr,
    input [4:0]     i_b_rd_addr,
    output reg [DATA_W-1:0] o_a_rd_data,
    output reg [DATA_W-1:0] o_b_rd_data,

    input          i_wr_en,
    input [4:0]     i_wr_addr,
    input [DATA_W-1:0] i_wr_data
);

    reg [DATA_W-1:0] regfile [31:0];
    integer i;

    always @(posedge i_clk or negedge i_rst_n) begin
        if (~i_rst_n) begin
            for (i=0; i<32; i=i+1) begin
                regfile[i] <= {DATA_W{1'b0}};
            end
        end else if (i_wr_en) begin
            regfile[i_wr_addr] <= i_wr_data;
        end
    end

    assign o_a_rd_data = regfile[i_a_rd_addr];
    assign o_b_rd_data = regfile[i_b_rd_addr];
endmodule
```

Figure 13: Register file module

The register file has 2 read ports and 1 write port. All registers are reset to 0, and all reads are completed on the same clock cycle.

ALU Module:

```

module alu #(
    parameter DATA_W = 32
)(
    input [DATA_W-1:0] i_opr_a,
    input [DATA_W-1:0] i_opr_b,
    input [5:0] i_op,
    output reg [DATA_W-1:0] o_res,
    output reg o_overflow
);

    reg [DATA_W:0] res;

    always @(*) begin
        res = {(DATA_W+1){1'b0}};
        o_res = {DATA_W{1'b0}};
        o_overflow = 1'b0;

        case (i_op)
            6'd1, 6'd3: begin
                res = {1'b0, i_opr_a} + {1'b0, i_opr_b};
                o_res = res[DATA_W-1:0];
                o_overflow = res[DATA_W];
            end
            6'd2: begin
                res = {1'b0, i_opr_a} - {1'b0, i_opr_b};
                o_res = res[DATA_W-1:0];
                o_overflow = res[DATA_W];
            end
            6'd4, 6'd5: begin
                res = {1'b0, i_opr_a} + {1'b0, i_opr_b};
                o_res = res[DATA_W-1:0];
                o_overflow = |res[DATA_W:8];
            end
            6'd6: o_res = i_opr_a & i_opr_b;
            6'd7: o_res = i_opr_a | i_opr_b;
            6'd8: o_res = ~(i_opr_a | i_opr_b);
            6'd9, 6'd10: begin
                res = {1'b0, i_opr_a} + {1'b0, i_opr_b};
                o_res = res[DATA_W-1:0];
                o_overflow = |res[DATA_W:12];
            end
            6'd11: o_res = (i_opr_a < i_opr_b) ? {(DATA_W-1){1'b0}}, 1'b1 : {DATA_W{1'b0}};
            default: begin
                res = {(DATA_W+1){1'b0}};
                o_res = {(DATA_W){1'b0}};
                o_overflow = 1'b0;
            end
        endcase
    end
endmodule

```

Figure 14: ALU module

All arithmetic and associated overflow detection is performed in the ALU. Addition results are calculated for opcode 1, 3, 4, 5, 9 and 10. For opcodes 1 & 3, the addition result is the result of an addition instruction and is written to the register. For opcodes 4 & 5, the addition result is the memory address associated with a load or store. Opcodes 9 & 10 are branch instructions, and the addition result is the branch relative offset value. The addition results are held in the 33 bit res signal, to determine if an overflow has occurred, various upper bits are checked to determine if an arithmetic overflow or invalid address has been calculated.

Testing:

The provided testbed.vp can be used to run the provided test patterns but is not useful for debugging. To solve this problem, I completed the provided testbed_temp.v file to run the provided testcases, dump waveforms in a .vcd file, and print debugging information. When the provided testcases passed, I created additional test patterns to check that overflow detection is working correctly. The passing testcases can be seen below.

```
=====
CPU stopped: status = 3 (EOF)
Total instructions executed: 56
=====

>>> ALL STATUS CHECKS PASSED! <<<
Simulation complete via $finish(1) at time 2295100 PS + 0
../00_TESTBED/testbed_temp.v:231 $finish;
```

Figure 15: Passing testbench for test pattern P0

```
=====
CPU stopped: status = 2 (OVERFLOW)
Total instructions executed: 32
=====

>>> ALL STATUS CHECKS PASSED! <<<
Simulation complete via $finish(1) at time 1345100 PS + 0
../00_TESTBED/testbed_temp.v:231 $finish;
```

Figure 16: Passing testbench for test pattern P1

```
=====
CPU stopped: status = 2 (OVERFLOW)
Total instructions executed: 3
=====

>>> ALL STATUS CHECKS PASSED! <<<
Simulation complete via $finish(1) at time 185100 PS + 0
../00_TESTBED/testbed_temp.v:231 $finish;
```

Figure 17: Passing testbench for test pattern P2

```
=====
CPU stopped: status = 2 (OVERFLOW)
Total instructions executed: 4
=====

>>> ALL STATUS CHECKS PASSED! <<<
Simulation complete via $finish(1) at time 225100 PS + 0
../00_TESTBED/testbed_temp.v:231 $finish;
```

Figure 18: Passing testbench for pattern P3

```
=====
CPU stopped: status = 2 (OVERFLOW)
Total instructions executed: 2
=====

>>> ALL STATUS CHECKS PASSED! <<<
Simulation complete via $finish(1) at time 145100 PS + 0
../00_TESTBED/testbed_temp.v:231          $finish;
```

Figure 19: Passing testbench for test pattern P1

Synthesis:

Synthesis was done using a dc_run.tcl file, very similar the provided Lab 3 file. To minimize power consumption, the synthesis script specified a target clock period of 9999ps, the maximum allowable clock period. Specifying a lower clock period would cause the synthesis tools to specify gates with higher power consumption.

clock i_clk (rise edge)	9999.00	9999.00
clock network delay (ideal)	0.00	9999.00
branch_taken_q_reg/CLK (DFFASRHQNx1_ASAP7_75t_L)	0.00	9999.00 r
library setup time	-17.14	9981.86
data required time		9981.86

data required time		9981.86
data arrival time		-1485.50

slack (MET)		8496.36

Figure 20: Synthesis timing report

With a clock period of 9999ps, data is required to be stable within 9981.86ps. The timing report indicates that data takes only 1485.5ps to be stable, for a slack of 8496.36ps. With this slack, the CPU can be run at a much higher frequency, or the design could be modified to fit more work into a single clock cycle.

Attributes
i - Including register clock pin internal power

Cell Internal Power = 159.4545 uW (97%)
Net Switching Power = 5.7032 uW (3%)

Total Dynamic Power = 165.1578 uW (100%)
Cell Leakage Power = 19.8988 uW

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	0.1541	0.0000	0.0000	0.1541	(83.29%)	i
register	6.1670e-04	1.1519e-04	3.0627e+06	3.7952e-03	(2.05%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	4.7038e-03	5.5880e-03	1.6836e+07	2.7128e-02	(14.66%)	

Figure 21: Synthesis Power report

The CPU uses a total of 165.1578 uW dynamic power, with 19.8988 uW leakage power, with most power consumed by the clock tree. Combinational logic elements consume far more power than sequential logic elements, with the majority of this power consumption likely occurring during addition operations.

```

Number of ports:                754
Number of nets:                 5765
Number of cells:                5102
Number of combinational cells:  3908
Number of sequential cells:     1188
Number of macros/black boxes:   0
Number of buf/inv:              627
Number of references:           38

Combinational area:             330.732721
Buf/Inv area:                   27.993600
Noncombinational area:          450.347037
Macro/Black Box area:           0.000000
Net Interconnect area:          undefined (No wire load specified)

Total cell area:                781.079759
Total area:                     undefined
1

```

Figure 22: Synthesis area report

The CPU has a cell area of 781.079759.